02. 구문 분석 과정 (Parsing Process)

개요

구문 분석(Parsing)은 토큰화 과정에서 생성된 토큰들을 의미 있는 구조인 추상 구문 트리(AST, Abstract Syntax Tree)로 변환하는 과정입니다. Monkey 언어의 파서는 Vaughan Pratt가 개발한 Pratt Parser 알고리즘을 사용하여 연산자 우선순위를 효율적으로 처리합니다.

1. Pratt Parser 개념과 Parser 구조체

1.1 Pratt Parser의 동작 원리

Pratt Parser는 **Top-Down Operator Precedence Parser**라고도 불리며, 다음과 같은 특징을 가집니다:

핵심 개념

- **연산자 우선순위 기반**: 각 연산자에 우선순위 값을 할당하여 파싱 순서 결정
- **재귀적 파싱**: 표현식을 재귀적으로 파싱하여 복잡한 중첩 구조 처리
- **함수 맵 기반**: 각 토큰 타입에 대응하는 파싱 함수를 맵으로 관리

Pratt Parser의 장점

1. **간결한 코드**: 복잡한 문법 규칙을 간단한 함수들로 표현
2. **확장성**: 새로운 연산자나 표현식을 쉽게 추가 가능
3. **효율성**: 불필요한 백트래킹 없이 한 번의 패스로 파싱 완료
4. **가독성**: 각 구문 요소별로 독립적인 파싱 함수 제공

1.2 Parser 구조체의 필드와 역할

```go
type Parser struct {
l *lexer.Lexer // 렉서 인스턴스
errors []string  // 파싱 에러 목록

currentToken token.Token // 현재 처리 중인 토큰
peekToken token.Token // 다음에 올 토큰 (lookahead)

prefixParseFns  map[token.TokenType]prefixParseFn // 전위 파싱 함수 맵
infixParseFns  map[token.TokenType]infixParseFn // 중위 파싱 함수 맵
}
```

각 필드의 상세 역할

1. 렉서 관련 필드

- `l *lexer.Lexer` : 토큰을 생성하는 렉서 인스턴스
- `currentToken` : 현재 파싱 중인 토큰
- `peekToken` : 다음 토큰을 미리 확인 (1-token lookahead)

2. 에러 처리 필드

- `errors []string` : 파싱 과정에서 발생한 모든 에러를 저장

3. 파싱 함수 맵

- `prefixParseFns` : 전위 표현식 파싱 함수들 (예: `-x`, `!true`)
- `infixParseFns` : 중위 표현식 파싱 함수들 (예: `x + y`, `x * y`)

1.3 Parser 초기화 과정

```go
func  New(l *lexer.Lexer) *Parser {
p := &Parser{l: l, errors: []string{}}


// 전위 파싱 함수 등록
p.prefixParseFns = make(map[token.TokenType]prefixParseFn)
p.registerPrefix(token.Identifier, p.parseIdentifier)
p.registerPrefix(token.Int, p.parseIntegerLiteral)
p.registerPrefix(token.Bang, p.parsePrefixExpression)
p.registerPrefix(token.Minus, p.parsePrefixExpression)
// ... 더 많은 전위 함수들


// 중위 파싱 함수 등록
p.infixParseFns = make(map[token.TokenType]infixParseFn)
p.registerInfix(token.Plus, p.parseInfixExpression)
p.registerInfix(token.Minus, p.parseInfixExpression)
p.registerInfix(token.LeftParen, p.parseCallExpression)
// ... 더 많은 중위 함수들


// 초기 토큰 설정 (2개 토큰을 미리 읽음)
p.nextToken()
p.nextToken()
```

```
return p
}
```

초기화 단계별 설명

1단계: 기본 구조체 생성

- 렉서 인스턴스와 빈 에러 슬라이스로 파서 생성

2단계: 파싱 함수 맵 초기화

- 각 토큰 타입에 대응하는 파싱 함수를 맵에 등록
- 전위 함수: 토큰이 표현식의 시작에 올 때 사용
- 중위 함수: 토큰이 두 표현식 사이에 올 때 사용

3단계: 토큰 버퍼 설정

- `currentToken` 과 `peekToken` 을 설정하여 1-token lookahead 구현
- 이를 통해 다음에 올 토큰을 미리 확인하여 파싱 결정 가능

2. AST 노드 시스템 분석

2.1 기본 인터페이스 구조

Monkey 언어의 AST는 3개의 핵심 인터페이스를 기반으로 구성됩니다:

Node 인터페이스 (최상위)

```
type Node interface {
TokenLiteral() string  // 노드와 연관된 토큰의 리터럴 값 반환
String() string  // 노드를 문자열로 표현 (디버깅용)
```

```
}
```

Statement 인터페이스 (문장)

```
type Statement interface {
Node
statementNode() // 마커 메서드 (문장임을 표시)
}
```

Expression 인터페이스 (표현식)

```
type Expression interface {
Node
expressionNode() // 마커 메서드 (표현식임을 표시)
}
```

2.2 주요 AST 노드들의 구조 분석

2.2.1 Program 노드 (루트 노드)

```
type Program struct {
Statements []Statement // 프로그램을 구성하는 문장들
}
```

- **역할**: AST의 최상위 노드로 전체 프로그램을 표현

- **구성**: 여러 개의 Statement들을 순서대로 저장

2.2.2 LetStatement (변수 선언문)

```
type LetStatement struct {
Token token.Token // 'let' 토큰
Name *Identifier // 변수명
Value Expression // 할당할 값
}
```

- **예시**: `let x = 5;`
- **구성 요소**:
- `Token` : `let` 키워드 토큰
- `Name` : 변수명을 나타내는 Identifier 노드
- `Value` : 할당할 값을 나타내는 Expression 노드

2.2.3 ReturnStatement (반환문)

```
type ReturnStatement struct {
Token token.Token // 'return' 토큰
ReturnValue Expression // 반환할 값
}
```

- **예시**: `return x + y;`
- **구성 요소**:
- `Token` : `return` 키워드 토큰

- `ReturnValue` : 반환할 값을 나타내는 Expression

2.2.4 ExpressionStatement (표현식 문장)

```
type ExpressionStatement struct {
Token token.Token // 표현식의 첫 번째 토큰
Expression Expression // 실제 표현식
}
```

- **예시**: `x + y;` (세미콜론으로 끝나는 표현식)
- **역할**: 표현식을 문장으로 감싸는 래퍼

2.2.5 Identifier (식별자)

```
type Identifier struct {
Token token.Token // 식별자 토큰
Value  string  // 식별자 이름
}
```

- **예시**: 변수명 `x` , 함수명 `add` 등
- **특징**: Expression 인터페이스를 구현 (값으로 평가 가능)

2.2.6 InfixExpression (중위 표현식)

```
type InfixExpression struct {
Token token.Token // 연산자 토큰 (예: +, -, *, /)
Left Expression // 왼쪽 피연산자
```

```
Operator  string  // 연산자 문자열
Right Expression // 오른쪽 피연산자
}
```

- **예시**: `x + y`, `a * b`, `foo == bar`
- **구조**: 이진 트리 형태로 연산자와 두 개의 피연산자 저장

2.2.7 PrefixExpression (전위 표현식)

```
type PrefixExpression struct {
Token token.Token // 전위 연산자 토큰 (예: !, -)
Operator  string  // 연산자 문자열
Right Expression // 피연산자
}
```

- **예시**: `-x`, `!true`
- **구조**: 단항 연산자와 하나의 피연산자

2.2.8 IfExpression (조건 표현식)

```
type IfExpression struct {
Token token.Token // 'if' 토큰
Condition Expression // 조건식
Consequence *BlockStatement // 참일 때 실행할 블록
Alternative *BlockStatement // 거짓일 때 실행할 블록 (선택적)
```

```
}
```

- **예시**: `if (x > y) { return x; } else { return y; }`
- **구성**: 조건식과 두 개의 실행 블록

2.2.9 FunctionLiteral (함수 리터럴)

```
type FunctionLiteral struct {
Token token.Token // 'fn' 토큰
Parameters []*Identifier // 매개변수 목록
Body *BlockStatement // 함수 본문
}
```

- **예시**: `fn(x, y) { return x + y; }`
- **구성**: 매개변수 리스트와 함수 본문 블록

2.2.10 CallExpression (함수 호출)

```
type CallExpression struct {
Token token.Token // '(' 토큰
Function Expression // 호출할 함수 (Identifier 또는 FunctionLiteral)
Arguments []Expression // 인수 목록
}
```

- **예시**: `add(1, 2)`, `fn(x) { x * 2 }(5)`
- **구성**: 함수 표현식과 인수 리스트

2.3 String() 메서드와 AST 출력 방식

모든 AST 노드는 `String()` 메서드를 구현하여 자신을 문자열로 표현할 수 있습니다.

2.3.1 LetStatement의 String() 구현

```
func (ls *LetStatement) String() string {
var  out bytes.Buffer

out.WriteString(ls.TokenLiteral() + " ") // "let "
out.WriteString(ls.Name.String()) // 변수명
out.WriteString(" = ") // " = "

if  ls.Value != nil {
out.WriteString(ls.Value.String()) // 할당값
}

out.WriteString(";") // ";"
return  out.String()
}
```

- **출력 예시**: `let x = 5;`

2.3.2 InfixExpression의 String() 구현

```
func (oe *InfixExpression) String() string {
```

```go
var out bytes.Buffer

out.WriteString("(") // "("
out.WriteString(oe.Left.String()) // 왼쪽 피연산자
out.WriteString(" " + oe.Operator + " ") // " + "
out.WriteString(oe.Right.String()) // 오른쪽 피연산자
out.WriteString(")") // ")"

return out.String()
}
```

- **출력 예시**: `(x + y)` (괄호로 우선순위 명시)

2.3.3 Program의 String() 구현

```go
func (p *Program) String() string {
var out bytes.Buffer

for _, s := range p.Statements {
out.WriteString(s.String()) // 각 문장을 순서대로 출력
}

return out.String()
}
```

- **역할**: 전체 프로그램을 하나의 문자열로 재구성
- **용도**: 디버깅, 테스트, 코드 생성 등에 활용

2.4 AST 노드 간의 관계

```
Program
├── LetStatement
│   ├── Identifier (Name)
│   └── Expression (Value)
├── ReturnStatement
│   └── Expression (ReturnValue)
└── ExpressionStatement
└── Expression
├── InfixExpression
│   ├── Expression (Left)
│   └── Expression (Right)
├── PrefixExpression
│   └── Expression (Right)
├── CallExpression
│   ├── Expression (Function)
│   └── []Expression (Arguments)
└── ...
```

이러한 트리 구조를 통해 복잡한 중첩 표현식도 체계적으로 표현할 수 있습니다.## 3.

연산자 우선순위와 파싱 함수 시스템

3.1 precedences 맵의 구조와 우선순위 정의

Pratt Parser의 핵심은 연산자 우선순위를 통한 파싱 순서 결정입니다.

3.1.1 우선순위 상수 정의

```
const (
_  int = iota
Lowest  // 가장 낮은 우선순위
Equals  // == 또는 !=
LessOrGreater  // < 또는 >
Sum  // +
Product  // *
Prefix  // -X 또는 !X
Call  // myFunction(X)
Index  // array[index]
)
```

3.1.2 precedences 맵 구조

```
var  precedences = map[token.TokenType]int{
token.Equal: Equals, // == 연산자
token.NotEqual: Equals, // != 연산자
token.LessThan: LessOrGreater, // < 연산자
```

```
token.GreaterThan: LessOrGreater, // > 연산자

token.Plus: Sum, // + 연산자

token.Minus: Sum, // - 연산자

token.Slash: Product, // / 연산자

token.Asterisk: Product, // * 연산자

token.LeftParen: Call, // 함수 호출 (

token.LeftBracket: Index, // 배열 인덱싱 [

}
```

3.1.3 우선순위 동작 원리

높은 숫자 = 높은 우선순위

- `Index` (9) > `Call` (8) > `Prefix` (7) > `Product` (6) > `Sum` (5) > `LessOrGreater` (4) > `Equals` (3) > `Lowest` (1)

예시: `2 + 3 * 4`

1. + 의 우선순위: Sum (5)
2. `*` 의 우선순위: Product (6)
3. Product > Sum이므로 `3 * 4` 를 먼저 파싱
4. 결과: `2 + (3 * 4)`

3.2 prefixParseFns와 infixParseFns 맵의 역할

3.2.1 파싱 함수 타입 정의

```
type (

prefixParseFn func() ast.Expression // 전위 파싱 함수
```

```
infixParseFn func(ast.Expression) ast.Expression // 중위 파싱 함수
)
```

3.2.2 prefixParseFns 맵 (전위 파싱 함수)

```
p.prefixParseFns = make(map[token.TokenType]prefixParseFn)
p.registerPrefix(token.Identifier, p.parseIdentifier) // 식별자
p.registerPrefix(token.Int, p.parseIntegerLiteral) // 정수 리터럴
p.registerPrefix(token.Bang, p.parsePrefixExpression) // ! 연산자
p.registerPrefix(token.Minus, p.parsePrefixExpression) // - 연산자
p.registerPrefix(token.True, p.parseBoolean) // true 키워드
p.registerPrefix(token.False, p.parseBoolean) // false 키워드
p.registerPrefix(token.LeftParen, p.parseGroupedExpression) // 괄호 표현식
p.registerPrefix(token.If, p.parseIfExpression) // if 표현식
p.registerPrefix(token.Function, p.parseFunctionLiteral) // 함수 리터럴
```

전위 파싱 함수의 특징:

- 매개변수 없음: 현재 토큰부터 시작하여 파싱
- 표현식의 시작 부분에 나타나는 토큰들을 처리
- 예시: `-5`, `!true`, `(x + y)`, `fn(x) { x }`

3.2.3 infixParseFns 맵 (중위 파싱 함수)

```
p.infixParseFns = make(map[token.TokenType]infixParseFn)
```

```go
p.registerInfix(token.Plus, p.parseInfixExpression) // + 연산자
p.registerInfix(token.Minus, p.parseInfixExpression) // - 연산자
p.registerInfix(token.Slash, p.parseInfixExpression) // / 연산자
p.registerInfix(token.Asterisk, p.parseInfixExpression) // * 연산자
p.registerInfix(token.Equal, p.parseInfixExpression) // == 연산자
p.registerInfix(token.NotEqual, p.parseInfixExpression) // != 연산자
p.registerInfix(token.LeftParen, p.parseCallExpression) // 함수 호출
p.registerInfix(token.LeftBracket, p.parseIndexExpression) // 배열 인덱싱
```

중위 파싱 함수의 특징:

- 매개변수 1개: 왼쪽 표현식을 받아서 처리
- 두 표현식 사이에 나타나는 토큰들을 처리
- 예시: `x + y`, `func(1, 2)`, `arr[0]`

3.3 parseExpression() 함수의 재귀적 파싱 과정

3.3.1 parseExpression() 함수 구조

```go
func (p *Parser) parseExpression(precedence int) ast.Expression {
// 1단계: 전위 파싱 함수 찾기
prefix := p.prefixParseFns[p.currentToken.Type]
if prefix == nil {
p.noPrefixParseFnError(p.currentToken.Type)
return nil
}
```

```
// 2단계: 전위 표현식 파싱
leftExp := prefix()

// 3단계: 중위 표현식 파싱 (우선순위 기반)
for !p.peekTokenIs(token.Semicolon) && precedence < p.peekPrecedence() {
infix := p.infixParseFns[p.peekToken.Type]
if  infix == nil {
return  leftExp
}

p.nextToken()
leftExp = infix(leftExp)
}

return  leftExp
}
```

3.3.2 파싱 과정 단계별 분석

1단계: 전위 파싱

- 현재 토큰에 대응하는 전위 파싱 함수를 찾아 실행
- 표현식의 시작 부분을 파싱하여 `leftExp` 생성

2단계: 중위 파싱 루프

- 조건 1: 세미콜론이 아닐 때 (`!p.peekTokenIs(token.Semicolon)`)
- 조건 2: 현재 우선순위보다 다음 토큰의 우선순위가 높을 때 (`precedence < p.peekPrecedence()`)
- 두 조건을 만족하면 중위 파싱 함수 실행

3단계: 재귀적 처리

- 중위 파싱 함수가 다시 `parseExpression()` 을 호출
- 더 높은 우선순위로 재귀 호출하여 오른쪽 표현식 파싱

3.3.3 파싱 과정 실제 예시

예시 1: `1 + 2 * 3` 파싱 과정

1. **초기 호출**: `parseExpression(Lowest)`

- 현재 토큰: `1` (Int)
- 전위 함수: `parseIntegerLiteral()` → `leftExp = 1`

2. **첫 번째 중위 루프**:

- 다음 토큰: `+` (Plus, 우선순위: Sum=5)
- `Lowest(1) < Sum(5)` → 루프 진입
- 중위 함수: `parseInfixExpression(leftExp=1)`

3. **parseInfixExpression() 내부**:

- 연산자: `+`
- 현재 우선순위: Sum(5)
- 오른쪽 표현식 파싱: `parseExpression(Sum)`

4. **재귀 호출**: `parseExpression(Sum)`

- 현재 토큰: `2` (Int)
- 전위 함수: `parseIntegerLiteral()` → `leftExp = 2`

- 다음 토큰: `*` (Asterisk, 우선순위: Product=6)
- `Sum(5) < Product(6)` → 루프 진입
- 중위 함수: `parseInfixExpression(leftExp=2)`

5. **두 번째 parseInfixExpression()**:

- 연산자: `*`
- 오른쪽 표현식: `parseExpression(Product)` → `3`
- 결과: `InfixExpression{Left: 2, Operator: "*", Right: 3}`

6. **최종 결과**:

- `InfixExpression{Left: 1, Operator: "+", Right: (2 * 3)}`

예시 2: `!-5` 파싱 과정

1. **초기 호출**: `parseExpression(Lowest)`

- 현재 토큰: `!` (Bang)
- 전위 함수: `parsePrefixExpression()`

2. **parsePrefixExpression() 내부**:

- 연산자: `!`
- 다음 토큰으로 이동: `-`
- 오른쪽 표현식: `parseExpression(Prefix)`

3. **재귀 호출**: `parseExpression(Prefix)`

- 현재 토큰: `-` (Minus)
- 전위 함수: `parsePrefixExpression()`
- 연산자: `-`
- 오른쪽 표현식: `parseExpression(Prefix)` → `5`

4. **최종 결과**:

- `PrefixExpression{Operator: "!", Right: PrefixExpression{Operator: "-", Right: 5}}`

3.4 우선순위 기반 파싱의 핵심 원리

3.4.1 우선순위 비교 메커니즘

```
func (p *Parser) peekPrecedence() int {
if p, ok := precedences[p.peekToken.Type]; ok {
return p
}
return Lowest
}

func (p *Parser) currentPrecedence() int {
if p, ok := precedences[p.currentToken.Type]; ok {
return p
}
return Lowest
}
```

3.4.2 파싱 결정 규칙

- **현재 우선순위 < 다음 우선순위**: 다음 연산자를 먼저 처리 (오른쪽 결합)
- **현재 우선순위 >= 다음 우선순위**: 현재 표현식 완료 후 상위로 반환

이러한 메커니즘을 통해 복잡한 표현식도 올바른 우선순위로 파싱할 수 있습니다.